CatBus Web Platform — 设计文档

> 负责人：前端工程师 域名：catbus.ai 目标：CatBus 网络的控制面板 + 公开展示页

一、产品定位

catbus.ai 是 CatBus 网络的”脸”。两类用户：

用户	需求	对应页面
访客（未注册）	了解 CatBus 是什么，怎么加入	Landing Page + 网络总览
节点所有者（已注册）	管理自己的 Agent、查看调用数据	Dashboard

二、页面结构

```
catbus.ai
│
├── / ........................... Landing Page（公开）
├── /network .................. 网络总览（公开）
├── /network/skills ........... Skill 目录（公开）
├── /network/nodes ............ 在线节点列表（公开）
│
├── /login .................... 登录（GitHub OAuth）
├── /dashboard ................ 我的面板（需登录）
├── /dashboard/agents ......... 我的 Agent 列表（需登录）
├── /dashboard/agents/:id ..... Agent 详情（需登录）
├── /dashboard/calls .......... 调用历史（需登录）
├── /dashboard/settings ....... 账号设置（需登录）
│
└── /docs ..................... 文档（公开，链接到 GitHub docs/）
```

三、各页面详细设计

3.1 Landing Page（/）

目标： 30 秒内让访客理解 CatBus 是什么，并想加入。

内容结构：

```
[Hero]
  标题：The Uber for AI Agents
  副标题：Your agent can call skills on other machines.
          Other agents can call yours.
  CTA按钮：Get Started → /docs    |    View Network → /network

[实时数据条]
  🟢 12 nodes online    |    🔧 47 skills available    |    📡 1,205 calls today

[How It Works — 三步图]
  1. Install → pip install catbus && catbus init
  2. Connect → catbus serve（你的 Agent 加入网络）
  3. Call → catbus call translate -i '{"text":"hello","target_lang":"zh"}'

[Demo GIF/视频]
  左右分屏：Agent A 发任务 → Agent B 执行 → 结果返回

[Use Cases — 三个卡片]
  🌐 跨机器调用：你的笔记本调用服务器上的 GPU 能力
  🤝 Agent 协作：不同 Agent 各有专长，互相帮忙
  🔌 零配置接入：装了 OpenClaw？一行命令加入网络

[Footer]
  GitHub | ClawHub | Docs | Discord
```

3.2 网络总览（/network）

目标： 实时展示 CatBus 网络的健康状况。

```
[顶部统计卡片]
  在线节点数    |    可用 Skill 数    |    今日调用量    |    平均响应时间

[节点地图]（可选，Phase 2）
  世界地图上标注在线节点位置（如果有地理信息的话）

[Top Skills 排行]
  排名    Skill 名称     Provider 数    今日调用量    平均延迟
  1       translate      3              156           148ms
  2       json_format    2              89            52ms
  3       echo           4              67            35ms
```

```
  ...

[最近活动流]（实时滚动）
  12:03:45  gouzai → translate → xiaohe (148ms) ✅
  12:03:42  xiaohe → echo → gouzai (35ms) ✅
  12:03:38  node-3 → json_format → xiaohe (52ms) ✅
  ...
```

3.3 Skill 目录（/network/skills）

目标： 浏览网络上所有可用的 Skill。

```
[搜索栏]
  🔍 Search skills...

[筛选]
  全部 | 在线 | 离线

[Skill 卡片列表]
  ┌──────────────────────────────────────────┐
  │ translate                                 │
  │ Translate text to target language          │
  │                                           │
  │ Providers: 3 online    Calls today: 156   │
  │ Avg latency: 148ms                        │
  │                                  [Try it] │
  └──────────────────────────────────────────┘

[Skill 详情弹窗 / 页面]
  - 描述
  - Input Schema（参数说明）
  - 在线 Provider 列表
  - 调用统计图表（7天/30天）
  - 示例调用命令
```

3.4 在线节点列表（/network/nodes）

目标： 查看谁在网络上。

```
[节点列表]
  节点名      Node ID      Skills 数    在线时长    状态
  gouzai      0df189...     4           2h 15m      🟢 在线
  xiaohe      6f72bc...     4           2h 10m      🟢 在线
  node-3      a1b2c3...     2           45m         🟢 在线
```

```
[点击展开]
  → 该节点提供的 Skill 列表
  → 最近处理的任务
  → 响应成功率
```

3.5 Dashboard — 我的面板（/dashboard）

需要登录。 用 GitHub OAuth 登录，绑定 node_id。

```
[概览卡片]
  我的 Agent 数     |     我提供的 Skill 数     |     被调用次数     |     调用他人次数

[我的 Agent 列表]（快捷入口）
  → gouzai （🟢 在线）   4 skills   被调用 89 次
  → xiaohe （🟢 在线）   4 skills   被调用 67 次

[最近调用]
  时间          方向       Skill        对方节点     耗时     状态
  12:03:45     ← 被调用    translate    node-3      148ms    ✅
  12:02:30     → 调用      echo         xiaohe      35ms     ✅
  ...
```

3.6 Agent 详情（/dashboard/agents/:id）

目标： 一个 Agent 的所有信息。

```
[Agent 信息]
  名称：gouzai
  Node ID：0df18901909a
  状态：🟢 在线（已连接 2h 15m）
  Server：relay.catbus.ai

[Skill 列表]
  ┌──────────────────────────────────────────┐
  │ Skill          状态      被调用次数    成功率   │
  │ translate      🟢 在线   89            100%   │
  │ text_stats     🟢 在线   45            98%    │
  │ json_format    🟢 在线   23            100%   │
  │ echo           🟢 在线   156           100%   │
  └──────────────────────────────────────────┘

[调用统计图表]
  - 7天调用量折线图（被调用 vs 调用他人）
  - 响应时间分布图
  - 成功率趋势
```

[调用历史]
完整的调用日志（分页，可筛选）

[配置建议]
💡 你的 translate skill 最受欢迎，考虑部署到更多节点
💡 你没有 image-compress skill，网络上有 5 个请求被拒

3.7 调用历史（/dashboard/calls）

[筛选]
方向：全部 | 被调用 | 调用他人
Skill：全部 | translate | echo | ...
状态：全部 | 成功 | 失败 | 超时
时间：今天 | 7天 | 30天 | 自定义

[列表]

时间	方向	Skill	对方	Agent	耗时	状态
03-10 12:03	← 被调用	translate	node-3	gouzai	148ms	✅
03-10 12:02	→ 调用	echo	xiaohe	gouzai	35ms	✅
03-10 11:58	← 被调用	translate	node-5	gouzai	152ms	✅
03-10 11:45	→ 调用	json_format	xiaohe	gouzai	52ms	✅

[点击展开]
→ 完整的 request input / output
→ 路由信息（经过哪个 relay）

3.8 账号设置（/dashboard/settings）

[账号信息]
GitHub 账号：xiaogong2000
邮箱：xxx@xxx.com

[绑定 Agent]
通过 node_id 绑定。流程：
1. 在本地跑 catbus status 获取 node_id
2. 在网页输入 node_id
3. 网页生成一个 验证 token
4. 本地跑 catbus verify <token> 完成绑定

[通知设置]
☐ Agent 离线时邮件通知
☐ 日报：每日调用统计
☐ 周报：每周网络报告

```
[API Key] (Phase 2)
  用于 API 访问 Dashboard 数据
```

四、数据来源

核心问题：网页怎么拿到数据？

4.1 Server 需要新增的 HTTP API

CatBus Server 目前只有 WebSocket 接口。需要加一组 HTTP API 给网页用：

```
GET /api/stats                    # 网络统计（节点数、skill 数、调用量）
GET /api/nodes                    # 在线节点列表
GET /api/nodes/:id                # 节点详情
GET /api/skills                   # skill 列表（聚合）
GET /api/skills/:name             # skill 详情
GET /api/skills/:name/providers   # 某 skill 的 provider 列表

# 以下需要认证
GET /api/me/agents                # 我的 agent 列表
GET /api/me/agents/:id            # 我的 agent 详情
GET /api/me/calls                 # 我的调用历史
POST /api/me/agents/bind          # 绑定 agent
POST /api/me/agents/verify        # 验证绑定
```

这部分 API 由我（后端）来加到 Server 上，前端工程师对接即可。

4.2 数据存储

目前 Server 全内存。网页需要的数据需要持久化：

数据	存储	说明
用户账号	PostgreSQL / SQLite	GitHub OAuth 信息
Agent 绑定关系	PostgreSQL / SQLite	user_id ↔ node_id
调用历史	PostgreSQL / SQLite	每次调用记录
实时节点状态	内存（Server）	通过 API 实时查
统计聚合	PostgreSQL / SQLite	定时聚合

我会在 Server 端加数据库和 API。前端只需要调 HTTP API。

4.3 实时数据

Landing Page 和 Network 页面需要实时更新的数据（在线节点数、最近活动流）。

两种方案：

- **方案 A（简单）：** 前端每 10 秒轮询 `/api/stats`
- **方案 B（更好）：** Server 提供一个 WebSocket 端点 `/api/ws/live`，推送实时事件

Phase 1 用方案 A，Phase 2 升级到方案 B。

五、技术选型建议

层	推荐	理由
框架	Next.js 或 Nuxt.js	SSR 对 SEO 好（Landing Page 需要）
UI	Tailwind CSS + shadcn/ui	快，好看
图表	Recharts 或 Chart.js	调用统计可视化
认证	NextAuth.js（GitHub OAuth）	简单
部署	Vercel 或 Cloudflare Pages	零运维
API 调用	fetch + SWR/React Query	自动缓存和刷新

前端工程师可以自选技术栈，以上仅为建议。

六、设计风格

品牌色

```
主色：#FF6B35（CatBus 橙 — 像龙猫巴士的颜色）
辅色：#1A1A2E（深色背景）
强调：#00D4AA（成功/在线 绿）
警告：#FF4444（离线/错误 红）
文字：#FFFFFF（亮色）/ #A0A0B0（次要）
```

风格参考

- Vercel Dashboard（简洁、数据密度高）
- Linear（现代、暗色主题）
- Cloudflare Radar（网络状态展示）

Logo

龙猫巴士 🚌 + 网络节点连接的意象。暂时用 emoji 🚌 代替。

七、开发分期

Phase 1（1-2 周）— 能看

- Landing Page（静态，硬编码数据也行）
- /network 页面（调 `/api/stats` 和 `/api/nodes`）
- /network/skills 页面
- 部署到 Vercel / Cloudflare Pages
- catbus.ai 域名指向

Phase 1 不需要登录，不需要数据库。 只展示公开数据，数据来源是 Server 的公开 API。

Phase 2（2-3 周）— 能用

- GitHub OAuth 登录
- Dashboard 我的面板
- Agent 列表 + 详情
- Agent 绑定流程
- 调用历史

Phase 3（持续）— 好用

- 调用统计图表（7天/30天趋势）
- 实时活动流（WebSocket）
- 节点地图

- 通知设置
- 移动端适配
- API Key 管理

八、前后端分工

任务	谁做	说明
Server HTTP API	我（后端）	加 `/api/*` 端点到 catbus-server
数据库 schema	我（后端）	users / agents / calls 表
认证后端	我（后端）	GitHub OAuth token 验证
Landing Page	前端工程师	静态页面 + 设计
Network 页面	前端工程师	调 API 展示
Dashboard 全部	前端工程师	调 API + 交互
部署配置	前端工程师	Vercel / Cloudflare

API 对接约定

- Base URL： `https://relay.catbus.ai/api/` （和 WebSocket 同域，Caddy 路由区分）
- 认证： `Authorization: Bearer <github_token>` 或 session cookie
- 格式：全部 JSON
- 错误格式： `{"error": {"code": "xxx", "message": "xxx"}}`
- 分页： `?page=1&limit=20` ，返回 `{"data": [...], "total": 100, "page": 1}`

我会先把 Phase 1 需要的 API 实现好，给前端一个 API 文档。前端先用 mock 数据开发，API 好了对接。

九、Mock API（前端先用这个开发）

在后端 API 准备好之前，前端可以用以下 mock 数据：

```
// mock/stats.json
{
  "online_nodes": 2,
  "total_skills": 8,
  "calls_today": 156,
  "avg_latency_ms": 148
}

// mock/nodes.json
{
  "data": [
    {
      "node_id": "0df18901909a",
      "name": "gouzai",
      "status": "online",
      "skills": ["translate", "text_stats", "json_format", "echo"],
      "uptime_seconds": 8100,
      "calls_handled": 89
    },
    {
      "node_id": "6f72bce2510d",
      "name": "xiaohe",
      "status": "online",
      "skills": ["translate", "text_stats", "json_format", "echo"],
      "uptime_seconds": 7800,
      "calls_handled": 67
    }
  ]
}

// mock/skills.json
{
  "data": [
    {
      "name": "translate",
      "description": "Translate text to target language",
      "providers": 2,
      "calls_today": 89,
      "avg_latency_ms": 148
    },
    {
      "name": "json_format",
      "description": "Format/beautify JSON string",
      "providers": 2,
      "calls_today": 45,
      "avg_latency_ms": 52
```

```
    },
    {
      "name": "text_stats",
      "description": "Count characters, words, lines in text",
      "providers": 2,
      "calls_today": 23,
      "avg_latency_ms": 35
    },
    {
      "name": "echo",
      "description": "Echo back the input (test connectivity)",
      "providers": 2,
      "calls_today": 156,
      "avg_latency_ms": 30
    }
  ]
}
```

十、检查清单

前端工程师开始前需要的

- ☐ 本文档（已提供）
- ☐ catbus.ai 域名 DNS 权限（你提供）
- ☐ GitHub repo 权限（你邀请 Collaborator）
- ☐ 设计风格确认（上面第六节，或者另出设计稿）
- ☐ Mock API 数据（上面第九节）
- ☐ 后端 API 文档（我后续提供，Phase 1 先用 mock）

前端工程师 Phase 1 交付物

- ☐ Landing Page 上线（catbus.ai）
- ☐ /network 页面（展示实时节点和 skill）
- ☐ /network/skills 页面
- ☐ /network/nodes 页面
- ☐ 移动端基本可用
- ☐ 部署到 Vercel / Cloudflare Pages